

Trust-Calibrated Strategy Search for NIFTY Weekly Options

Hardik Kalia Manasa Kalaimalai Aanvik Bhatnagar Keval Jain

A constrained LLM-assisted search system for real options data

Real dataset
448 dates

Live model
Qwen 14B

Selected test
+359.6 pts

Proof-of-concept scope. We built the end-to-end loop: local LLM proposals, strict validation, Python-side tuning, fixed-contract option backtesting with costs, and held-out reporting. The live LLM budget was intentionally small; the system is designed to scale when stronger models and more compute are available.

The full loop: from raw data to trust update



Frozen data

Local NSE bhavcopy files;
no live scraping inside
search.

Strict interface

JSON rules only; no exe-
cutable trading code from
LLM.

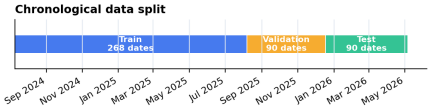
Held-out reporting

Train tunes, validation
ranks, test reports.

Data universe and allowed features

Dataset snapshot

- ▶ 448 trading dates
- ▶ 2024-07-10 to 2026-05-05
- ▶ 340,776 contract-level NIFTY option rows
- ▶ 3-way chronological split: train, validation, test



Feature	Intuition
atm_iv_proxy	ATM option premium as a rough volatility proxy
iv_minus_rv20	option-vol proxy minus realized volatility
rv20	20-day realized volatility
skew_proxy	relative OTM put/call pricing
term_slope_proxy	near-vs-next expiry premium slope
oi_change	ATM CE+PE open-interest change
volume_proxy	ATM CE+PE trading volume
underlying_ret1/5	Short-term NIFTY momentum

The LLM side: simple on purpose

Live-simple prompt contract

```
Return exactly one JSON object:  
{"strategies": [...]}  
Use only approved templates/features.  
Use formula_type="single_condition".  
Use exactly one condition.  
Use hold_days="tune".  
No thresholds. No Python code.
```

Proposal modes

- ▶ **idea**: fresh structures
- ▶ **mutation**: variants of prior winners
- ▶ **diversity**: different templates/features

A valid output

```
{"name": "atm_straddle_oi_change",  
 "template": "ATM_STRADDLE",  
 "side": "long",  
 "formula_type": "single_condition",  
 "conditions": [{"feature": "oi_change", "op": ">"}],  
 "score_terms": [],  
 "hold_days": "tune"}
```

A real failure mode

```
{"feature": "rv20", "op": ">"}
```

Invalid JSON. Rejected before tuning/backtest.

Validation gate: the system does not blindly trust the model

LLM Audit

The system does not blindly trust LLM output: every candidate is parsed, schema-validated, deduplicated, tuned, and scored by Python.

Generated

13

Accepted

10

Rejected

3

Acceptance rate

77%

Top Rejection Reasons

Reason	Count
invalid JSON: Expecting ',' delimiter	2
duplicate proposal signature	1

Proposal Mode Breakdown

Cycle	Mode	Accepted	Count
1	diversity	☒	1
1	idea	☒	2
1	mutation	☒	1
2	diversity	☒	1
2	idea	☐	1
2	mutation	☐	1
3	diversity	☐	1
3	diversity	☒	2
3	idea	☒	2
3	mutation	☒	1

Accepted And Rejected Examples

atm_straddle_of_change

Rejected: unnamed candidate

Reason: invalid JSON: Expecting ',' delimiter

Final live run

- ▶ 9 Qwen 14B proposal batches
- ▶ 23 parsed candidate objects
- ▶ 21 accepted by validation
- ▶ 2 malformed JSON proposals rejected
- ▶ 1 duplicate rejected
- ▶ 10 strategies tuned and evaluated

Validation checks

JSON shape, required fields, feature names, operators, template-side pairs, DTE limits, duplicates.

Python-side tuning and scoring

Candidate = structure + parameters

$(s, \theta) = \text{template} + \text{signal structure, tuned by Python}$

- ▶ Thresholds are tuned as **training-window quantiles**
- ▶ Grid over DTE windows, hold days, strike offset, and thresholds
- ▶ Validation-trade eligibility prevents sparse rules from dominating
- ▶ Standard, expanding walk-forward, and rolling leaderboards are saved

Selection score

- ▶ reward: validation Sharpe + bounded average PnL
- ▶ penalize: drawdown, turnover, complexity
- ▶ reliability: validation trade-count penalty

Test metrics are reported only after selection.

Three validation views

- ▶ standard chronological leaderboard
- ▶ expanding walk-forward leaderboard
- ▶ rolling validation leaderboard

Backtesting: close the same option contracts

Entry

Signal on day t ; enter next trading date.
For a straddle, choose actual ATM CE and PE contracts with a specific expiry and strike.

Exit

After tuned hold days, close the same expiry, strike, and CE/PE contracts. If the exact exit date is missing, use nearest available same-contract quote after entry.

Cost

Default transaction cost = $0.002 \times$ gross entry premium per trade, i.e. 0.20%.

Step

Example: long ATM straddle

Signal

$oi_change > 0.7245$

Entry

Buy ATM CE + buy ATM PE

Contract

same expiry E , strike K , types CE/PE

Exit

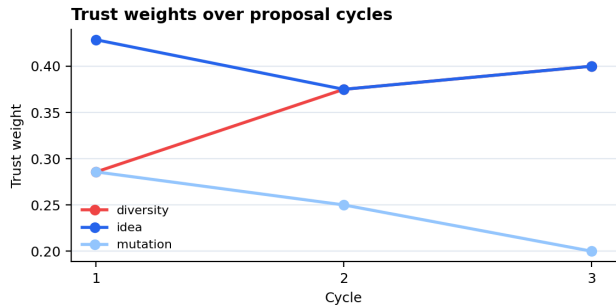
sell same E, K , CE/PE after 2 days

PnL

exit value - entry value - cost

This is a key realism upgrade: the backtest does not reselect a new ATM option at exit.

Trust calibration across proposal modes



How trust works

- ▶ Each mode has reliability state
- ▶ Strong validation candidates increase trust
- ▶ Weak candidates reduce relative weight
- ▶ Exploration floor keeps all modes alive

Final cycle

Idea: 0.40 Diversity: 0.40

Mutation: 0.20

Selected strategy: ATM straddle on OI change

Rule

Buy a long ATM straddle when ATM option open-interest change is high.

Item

Value

Template

ATM_STRADDLE,
long

Signal

oi_change > 0.7245

DTE

3–10

Hold

2 trading days

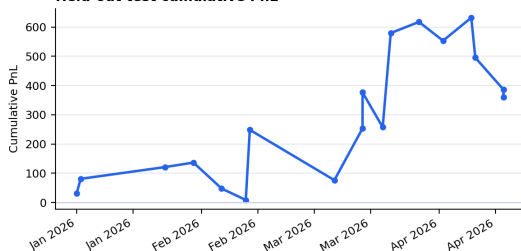
Threshold

0.80 quantile

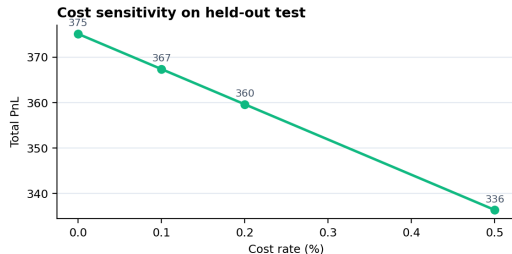
PnL is in option-premium points for one unscaled strategy unit, after 0.20% default cost.

Split	Trades	PnL	Sharpe	DD
Train	33	597.7	1.14	-298.3
Validation	12	122.9	0.46	-128.5
Test	18	359.6	0.65	-272.5

Held-out test cumulative PnL



Robustness: costs, folds, and test-only winners



The selected strategy remains positive across reported cost rates.

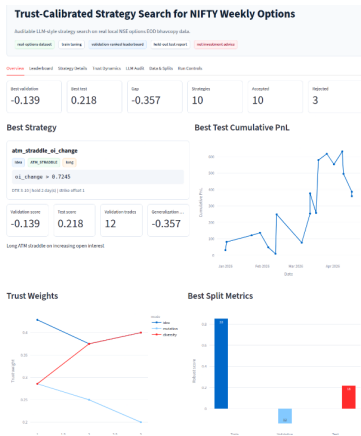
Leaderboard comparison

Strategy	Std	WF	Roll	Test PnL
ATM Straddle OI	1	9	9	359.6
Call Spread Skew	2	7	6	-505.0
Put Spread IV	3	3	3	385.4
Put Spread Term	4	5	5	218.9
Risk Rev. RV20	5	8	8	5600.2

Why this matters

Some strategies made large test-period PnL, but were not selected because validation ranking was weaker. This prevents test-period cherry-picking.

What the dashboard makes visible



The app is the audit trail

- ▶ Leaderboards and selected strategy
- ▶ LLM trace and rejection reasons
- ▶ Trust weights across cycles
- ▶ Trade-level details and cost sensitivity
- ▶ Train / validation / test split information

The live LLM path is intentionally limited, but the rest of the research loop is fully reproducible from saved result files.

Why this needs a controlled loop

Proof-of-concept scope

We built this as an MVP to prove the full loop works:

LLM proposal → validation → tuning → backtest →

The live LLM run was intentionally small because local models were slow and weaker models often produced schema errors.

Our design choice

The LLM is only a proposal engine. It suggests a readable strategy structure, but Python decides whether it is valid, how it is tuned, and how it performs on historical data.

Why not just ask an LLM?

- ▶ Invalid JSON or missing fields
- ▶ Unknown features
- ▶ Bad option template combinations
- ▶ Untestable explanations

Why options are harder Every trade must specify and later close the same:

- ▶ expiry, strike, CE/PE type
- ▶ entry date, exit date, cost

Takeaways and future scale

What we delivered

- ▶ Real local NSE options dataset and strategy panel
- ▶ Live Qwen 14B proposals through a strict JSON interface
- ▶ Validation, quantile tuning, same-contract backtesting
- ▶ Trust-calibrated allocation across idea/mutation/diversity
- ▶ Cost sensitivity and held-out test reporting

What stronger LLMs + hardware unlock

- ▶ Natural-language user constraints
- ▶ Hundreds of candidates and more rounds
- ▶ Direct comparison between trust policies
- ▶ Richer validated grammars, still not arbitrary code
- ▶ More complete risk, lot-size, and capital modeling

Bottom line: LLMs can propose candidate structures, but deterministic validation and backtesting decide what survives.